

Exp No.: 3 Implementing Asset Management with Hyperledger Fabric

Aim

To implement an Asset Management application using Hyperledger Fabric by creating, reading, updating, transferring, and deleting assets through JavaScript chaincode and verifying the transactions on the blockchain ledger.

Software Requirements

- Kali Linux / Ubuntu
- Docker
- Docker Compose
- Node.js (Version 18 or later)
- npm
- Git
- Hyperledger Fabric Samples
- Hyperledger Fabric Binaries
- Hyperledger Fabric Docker Images
- Visual Studio Code (Optional)

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

Procedure

Step 1: Navigate to the Fabric Test Network

Open the terminal and navigate to the Hyperledger Fabric test network.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

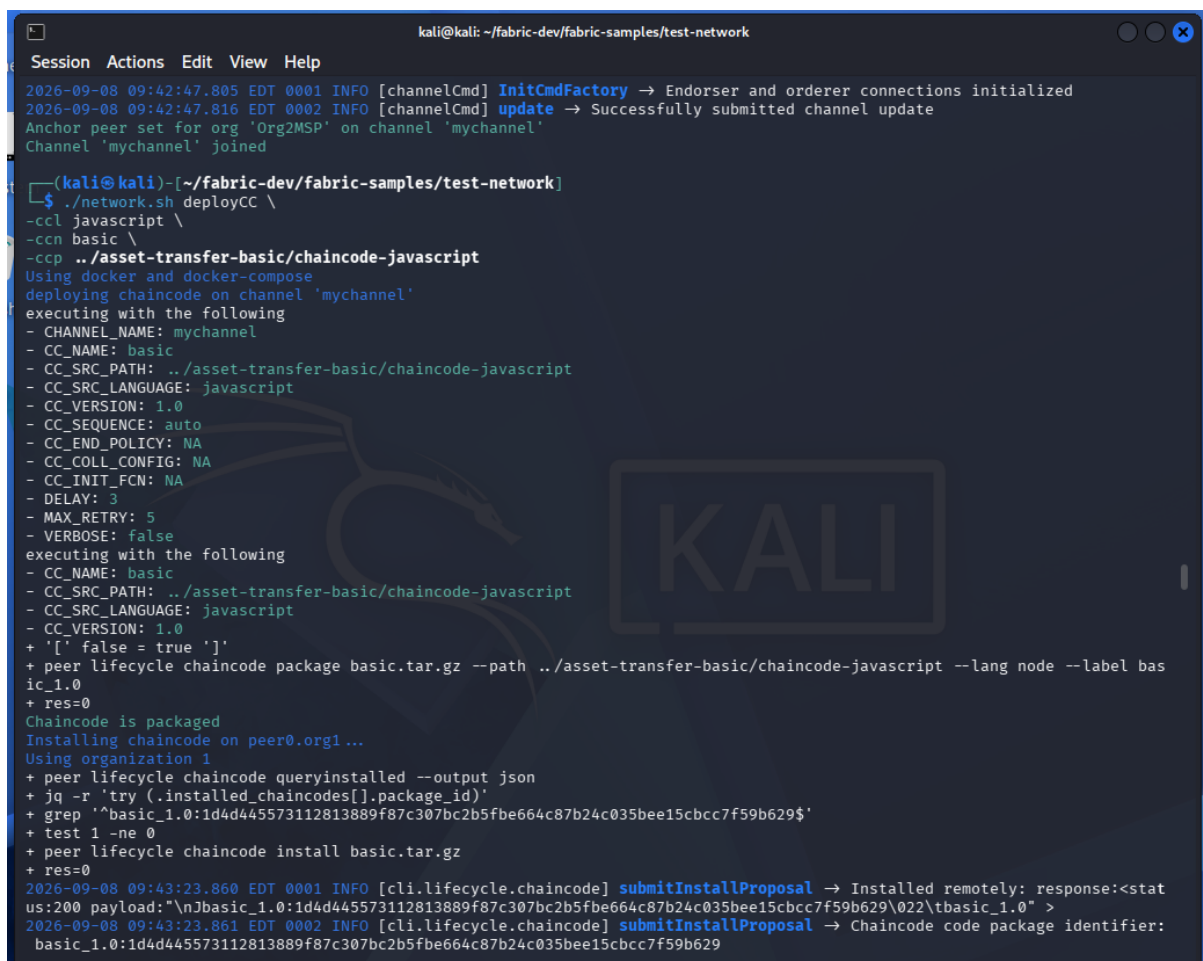
Start the Fabric network and create the application channel.

```
./network.sh up createChannel -ca
```

Step 3: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer chaincode.

```
./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript
```



```
kali@kali: ~/fabric-dev/fabric-samples/test-network
Session Actions Edit View Help
2026-09-08 09:42:47.805 EDT 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
2026-09-08 09:42:47.816 EDT 0002 INFO [channelCmd] update → Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
(kali@kali) [~/fabric-dev/fabric-samples/test-network]
$ ./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep "basic_1.0:1d4d445573112813889f87c307bc2b5fbc664c87b24c035bee15cbcc7f59b629$"
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
+ res=0
2026-09-08 09:43:23.860 EDT 0001 INFO [cli.lifecycle.chaincode] submitInstallProposal → Installed remotely: response:<status:200 payload:"\njbasic_1.0:1d4d445573112813889f87c307bc2b5fbc664c87b24c035bee15cbcc7f59b629\022\tbasic_1.0" >
2026-09-08 09:43:23.861 EDT 0002 INFO [cli.lifecycle.chaincode] submitInstallProposal → Chaincode code package identifier: basic_1.0:1d4d445573112813889f87c307bc2b5fbc664c87b24c035bee15cbcc7f59b629
```

Figure 1: Deploying the asset-transfer-basic JavaScript chaincode

Step 4: Initialize the Ledger

Populate the ledger with sample assets.

```
peer chaincode invoke ... -c '{"function":"InitLedger","Args":[]}'
```

Step 5: Query All Assets

Retrieve all asset records stored in the blockchain.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

Step 6: Create a New Asset

Create a new asset named asset11.

```
peer chaincode invoke ... -c  
'{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}'
```

Step 7: Read the Asset Details

Retrieve the information of asset11.

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset11"]}'
```

Step 8: Update Asset Information

Modify the asset's color, size, and appraised value.

```
peer chaincode invoke ... -c  
'{"function":"UpdateAsset","Args":["asset11","Black","15","Arun","1200"]}'
```

```
kali@kali: ~/fabric-dev/fabric-samples/test-network

Session Actions Edit View Help

er":{"Jin Soo","Size":10,"docType":"asset"},{"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem"
2026-09-08 09:56:49.998 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:
200 payload:{"ID":"asset11","Color":"White","Size":12,"Owner":"Arun","AppraisedValue":900}

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset11"]}'
{"AppraisedValue":900,"Color":"White","ID":"asset11","Owner":"Arun","Size":12}

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function":"UpdateAsset","Args":["asset11","Black","15","Arun","1200"]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$PWD/organizations/peerOrganizations/org1.example.com/tlsca/tlsca.org1.example.com-cert.pem" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem"
2026-09-08 09:57:33.423 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:
200 payload:{"type":"Buffer","data":[]}

(kali@kali)-[~/fabric-dev/fabric-samples/test-network]
$ peer chaincode query \
-C mychannel \
-n basic \
```

Figure 2: Creating, reading, and updating asset11 on the ledger

Step 9: Transfer Asset Ownership

Transfer ownership of asset11 to another user.

```
peer chaincode invoke ... -c  
'{"function":"TransferAsset","Args":["asset11","David"]}'
```

Step 10: Verify the Updated Asset

Query asset11 again to verify the updated owner.

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset11"]}'
```

Step 11: Delete the Asset

Remove asset11 from the ledger.

```
peer chaincode invoke ... -c '{"function":"DeleteAsset","Args":["asset11"]}'
```

Step 12: Verify Asset Deletion and Stop the Network

Attempt to read the deleted asset, confirm it no longer exists, then shut down the network.

```
peer chaincode query -C mychannel -n basic -c  
'{"Args":["ReadAsset","asset11"]}'  
./network.sh down
```

```
kali@kali: ~/fabric-dev/fabric-samples/test-network

Session Actions Edit View Help

--tlsRootCertFiles "$PWD/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem"
2026-09-08 09:59:07.682 EDT 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:
200 payload:{"type":"Buffer","data":[]}"

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset11"]}'
Error: endorsement failure during query. response: status:500 message:"The asset asset11 does not exist"

(kali@kali)~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["ReadAsset","asset11"]}'
Error: endorsement failure during query. response: status:500 message:"The asset asset11 does not exist"

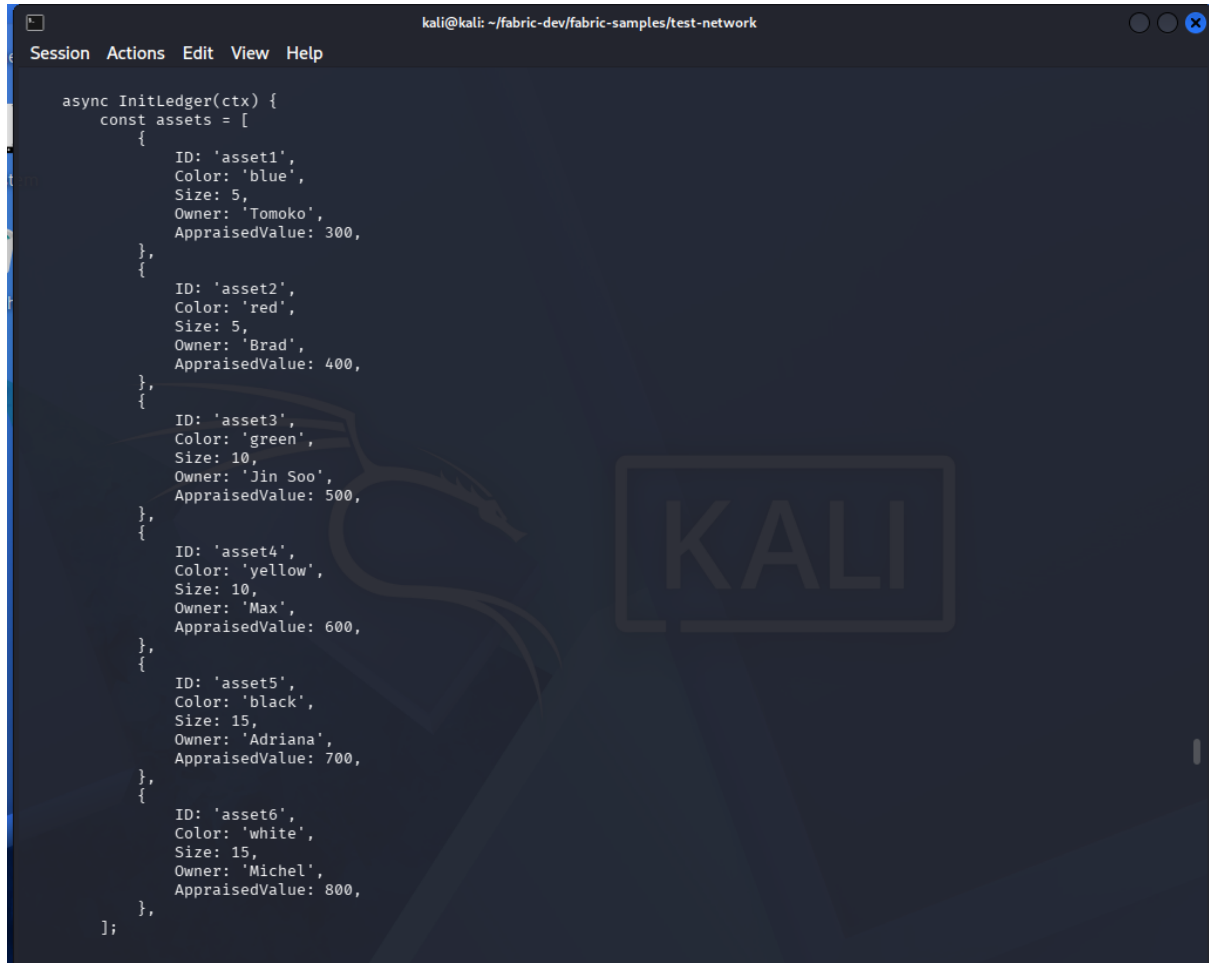
(kali@kali)~/fabric-dev/fabric-samples/test-network
$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] Running 10/10
✓ Container orderer.example.com Removed 0.6s
✓ Container peer0.org1.example.com Removed 1.0s
✓ Container ca_org1 Removed 10.8s
✓ Container peer0.org2.example.com Removed 0.9s
✓ Container ca_orderer Removed 10.5s
✓ Container ca_org2 Removed 10.6s
✓ Network fabric_test Removed 0.3s
✓ Volume compose_peer0.org1.example.com Removed 0.0s
✓ Volume compose_peer0.org2.example.com Removed 0.0s
✓ Volume compose_orderer.example.com Removed 0.0s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0
ff0cc1497615f5e33a8bfa908e0fd3555f864cfa1398b52d341cb:latest
Deleted: sha256:3dc540b1bf291598d2e499a429e8c65a6b71356def621334a769f0cdfa77616e
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9
747cc8433ac0e74bfaade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:7bd3c0c3584d61e2eef17e5d7aeeef0738edcfd95881064a16becba4119386fc

(kali@kali)~/fabric-dev/fabric-samples/test-network
$
```

Figure 3: Verifying asset deletion (asset11 does not exist) and stopping the network

Step 13: Examine the Chaincode Source

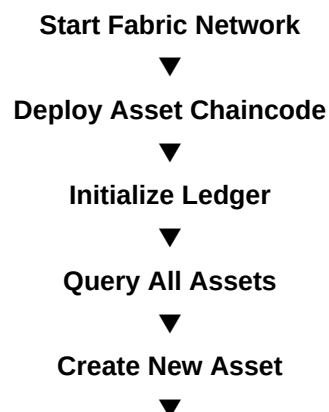
Review the InitLedger function in the JavaScript chaincode to see the predefined assets.

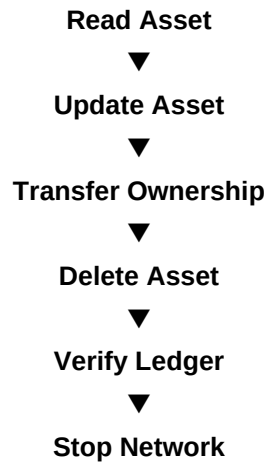


```
async InitLedger(ctx) {
  const assets = [
    {
      ID: 'asset1',
      Color: 'blue',
      Size: 5,
      Owner: 'Tomoko',
      AppraisedValue: 300,
    },
    {
      ID: 'asset2',
      Color: 'red',
      Size: 5,
      Owner: 'Brad',
      AppraisedValue: 400,
    },
    {
      ID: 'asset3',
      Color: 'green',
      Size: 10,
      Owner: 'Jin Soo',
      AppraisedValue: 500,
    },
    {
      ID: 'asset4',
      Color: 'yellow',
      Size: 10,
      Owner: 'Max',
      AppraisedValue: 600,
    },
    {
      ID: 'asset5',
      Color: 'black',
      Size: 15,
      Owner: 'Adriana',
      AppraisedValue: 700,
    },
    {
      ID: 'asset6',
      Color: 'white',
      Size: 15,
      Owner: 'Michel',
      AppraisedValue: 800,
    },
  ],
};
```

Figure 4: InitLedger function showing the predefined sample assets in the chaincode

Flow of Asset Management





Result

The Asset Management application was successfully implemented using Hyperledger Fabric. The asset lifecycle operations, including asset creation, querying, updating, ownership transfer, and deletion, were executed successfully. The transactions were recorded on the blockchain ledger, demonstrating secure and immutable asset management in a permissioned blockchain network.